

TemaFinale26 – Sprint 0: Analisi dei Requisiti

Requirements

Si riporta il **testo esatto del committente** (Protobook, cap. 31 – TemaFinale26):

"A Maritime Cargo shipping company (from now on, simply company) intends to automate the operations of load of containers in the ship's cargo hold (or simply hold). To this end, the company plans to employ a Differential Drive Robot (from now, called cargorobot). The hold is a rectangular, flat area with an Input/Output port (IOPort). The area provides 4 slots to store the containers and a slot named slot5."

- "• The slots1-4 depict the hold areas reserved to store one container each.*
- The slot5 depicts an area, where the cargorobot must temporarily store a container, before to place it in one of the slots1-4. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed.*
- The IOPort is a device with a pushbutton and a display. The pushbutton is pressed by the customer in order to send a request to load a container on the cargo. The display is used to show the answer to the request and to show the current state of the hold.*
- The sensor associated to the IOPort is a device (a sonar) used to detect the presence of a container, when it measures a distance D , such that $D < DFREE/2$, during a reasonable time (e.g. 3 secs)."*

TF2026 Requirements – testo esatto:

"The company asks us to build service named cargoservice that should work as follows. The cargoservice is able to receive a request to load a container sent by some customer by using the pushbutton of the IOPort.

- It sends the answer retrylater, if the IOPort is currently occupied by a container or if the system is Out of service.*
- It rejects the request when the hold is already full, i.e. the slots1-4 are already occupied.*
- Otherwise, it considers the system as engaged, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led. When the request to load is accepted, the customer must move the container in the sensor area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes disengaged. Then, the cargoservice uses the cargorobot to move the container from the IOPort to the slot5 (for marking the container) and then to the reserved slot. The service must also show on the display on the IOPort:*
 - the current state of the hold;*
 - the message 'Service working', when it is all is going well;*
 - the message 'Out of service' if the sonar sensor measures a distance $D > DFREE$ for at least 3 secs (perhaps a failure of the sonar)."*

Layout dell'area (dalla figura allegata al tema)

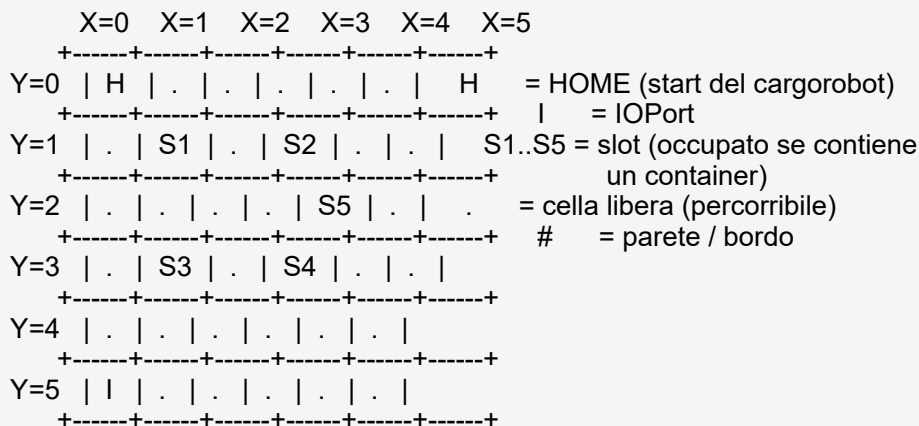
La figura del tema e' **illustrativa**: mostra le posizioni relative ma non formalizza l'area.



Attenzione: IOPort e sensor **non** sono un unico dispositivo. Il testo del committente dice "the IOPort is a device with a pushbutton and a display" e, separatamente, "the *sensor associated to the IOPort*": il sensor e' un dispositivo **distinto**, associato all'IOPort. (Il [SONAR] in alto nella figura e' l'eventuale sonar del **robot**, estraneo all'applicazione — cfr. P2.)

Formalizzazione dell'area

La figura sopra **non formalizza** l'area. Per ragionare in modo non ambiguo (e per rendere il test **automatizzabile**) l'area va rappresentata come una **matrice di celle**: ogni cella ha coordinate (X,Y), una mossa del cargorobot porta a una cella adiacente, e lo stato della stiva e' definito da quali celle-slot sono occupate.



Forma 'machine-readable' (convenzione del docente, una riga per @, 0=libera 1=occupata):
es. 100000@010100@000010@010100@000000@100000

Lo schema sopra e' **schematico** (coerente con la figura): mostra il **metodo** di formalizzazione (griglia a coordinate + simboli), non i valori definitivi. Le coordinate **autorevoli** di HOME/IOPort/slot **non sono inventate**: provengono dalla **mappa built-in** del servizio DDR (cfr. **D1**): le coordinate effettive di HOME, IOPort e slot vanno **lette da quella mappa**, non fissate qui.

Requisiti come user stories

Su indicazione del docente, i requisiti vengono espressi anche come **user stories** (formato "*Come <ruolo>, voglio <obiettivo>, cosi' che <beneficio>*"), per metterne a fuoco gli attori e il valore.

Sono una vista **complementare** al testo del committente, non una sua sostituzione.

US1. Come **customer**, voglio inviare una richiesta di carico premendo il pushbutton dell'IOPort, così da poter caricare un container sulla nave.

US2. Come **customer**, voglio ricevere sul display la risposta alla richiesta (slot riservato | retrylater | reject), così da conoscerne l'esito.

US3. Come **committente**, voglio che la richiesta sia **rifiutata** quando la stiva è piena (slot1-4 occupati), così da non accettare carichi impossibili.

US4. Come **committente**, voglio che il sistema risponda **retrylater** quando l'IOPort è occupato da un container o è Out of service, così da gestire la richiesta più tardi.

US5. Come **customer**, quando la richiesta è accettata voglio depositare il container nell'area del sensor entro un tempo prefissato (~30s), altrimenti il sistema si disengagia.

US6. Come **committente**, voglio che il cargorobot porti il container dall'IOPort allo slot5 (marcatura) e poi allo slot riservato, così che ogni container sia etichettato e immagazzinato correttamente.

US7. Come **committente**, voglio che al termine il cargorobot torni a HOME, così che il servizio sia pronto per una nuova richiesta.

US8. Come **operatore**, voglio vedere sul display lo stato corrente della stiva e i messaggi "Service working"/"Out of service", così da conoscere lo stato del sistema.

US9. Come **committente**, voglio che il sistema faccia lampeggiare un LED mentre è *engaged*, così che l'impegno del sistema sia visibile.

US10. Come **committente**, voglio che il sistema vada "Out of service" se il sonar misura $D > D_{FREE}$ per $\geq 3s$ (possibile guasto), così da non operare con un sensore guasto, riprendendo quando $D \leq D_{FREE}$.

Analisi dei requisiti

Goal: formalizzare le entità del dominio escludendo le ambiguità del linguaggio naturale (dichiarazioni Java per il dominio + modello qak per le interazioni).

Entità formalizzate

Container — merce da caricare; nel TF26 non ha peso né PID a monte: riceve un **barcode** solo durante la marcatura in slot5.

Slot — area di stoccaggio della stiva (slot1..slot4 di carico; slot5 di marcatura):

```
public class Slot {
    String name;           // "slot1".. "slot4" | "slot5"
    int   posX, posY;      // posizione nella mappa
    boolean occupied;
    boolean isMarkingArea; // true solo per slot5
}
```

Hold — aggregato degli slot di carico (slot1..4) + slot5 di marcatura:

```
public class Hold {
    Slot[] slots;           // slot1..slot4 (+ slot5 marcatura)
    boolean ioportOccupied(); // container presente all'IOPort
    boolean isFull();        // tutti gli slot1..4 occupati
    String reserveFreeSlot(); // nome di uno slot libero, "" se pieno
    void   confirmStored(String slotName);
    void   freeSlot(String slotName);
}
```

IOPort — dispositivo con **pushbutton** (input richiesta) e **display** (output: risposta + stato hold). Il **sensor** che rileva il container e' un dispositivo **separato associato** all'IOPort, non parte di esso (testo del committente: "the sensor *associated to* the IOPort").

Sensor dell'IOPort (sonar) — misura la distanza D di un oggetto davanti all'IOPort. Condizioni fissate dal committente:

```
container presente : D < DFREE/2 per ~3 s
sospetto guasto   : D > DFREE per ≥ 3 s
```

Da chiarire col committente: cosa sia esattamente questo sensor e in che modo fornisca il dato (i requisiti non lo specificano). Va inoltre tenuto **distinto** dall'eventuale sonar montato sul **robot** (es. in WEnv): quest'ultimo non ha alcun ruolo nel cargoservice.

Marker — dispositivo in slot5 che etichetta il container con un barcode e **segnala** il completamento della marcatura. (Il significato esatto di "segnala" va **chiarito col committente**; al cargoservice basta il segnale di completamento.)

LED — **indicatore di stato** (non un attuatore) che **lampeggia** mentre il sistema e' *engaged*. La sua **collocazione non e' specificata** dai requisiti (cfr. domanda aperta).

CargoRobot — il DDR fornito dal committente (risorsa data) che raggiunge una posizione-obiettivo su comando.

Da analizzare. Il committente/azienda dispone di *piu' software* per il DDR (un POJO come RobotObj26 e servizi a messaggi come RobotService26 o il pianificatore robotsmart): **quale** usare per realizzare il cargorobot non e' un requisito, ma va **analizzato e motivato** (vedi Analisi del Problema, Sprint 1), non solo rinviato al progetto.

Macro-parti del sistema

Modello delle macro-parti, con indicazione di cosa e' **fornito dal committente** e cosa va **sviluppato** (richiesto dallo Sprint0, cap. 32.1.1):

Macro-parte	Fornito / da sviluppare	Natura software
cargoservice	da sviluppare	logica applicativa (servizio a messaggi)
cargorobot (DDR)	fornito dal committente	POJO o servizio gia' esistente — <i>quale</i> usare e' una scelta di progetto
sensor dell'IOPort	fornito (hardware) / simulato	sorgente di dati di distanza; natura/trasporto da chiarire
pushbutton + display	forniti (IOPort) / simulati	input richiesta / output (display: pagina web)
marker (slot5)	fornito / simulato	segnala il completamento della marcatura
LED	fornito (hardware) / simulato	indicatore di stato (engaged)

Quanti di questi componenti siano realizzati come servizi autonomi, con **quale** tecnologia e **quale** trasporto, e' una decisione della fase di progetto da motivare: non e' un requisito.

Un modello (astratto) delle interazioni

A livello di requisiti interessa **quali** informazioni scorrono tra i componenti, non **come** vengano realizzate. Il **cargoservice** e' il componente centrale:

- riceve dal **pushbutton** una richiesta di carico e ne restituisce una risposta (slot riservato | retry|later | reject);
- aggiorna il **display** con lo stato corrente della stiva e i messaggi di servizio;
- e' informato dal **sensor** dell'IOPort della presenza del container e di un eventuale guasto;
- e' informato dal **marker** del completamento della marcatura;
- comanda il **cargorobot** a raggiungere una posizione;
- attiva il **LED** mentre il sistema e' *engaged*.

L'interazione **request/reply** del pushbutton e' gia' un **requisito** e viene **formalizzata in qak** (a livello di interfaccia dei messaggi) nell'[Analisi del Problema \(Sprint 1\)](#); anche le user stories US1..US10 prefigurano il primo test plan.

Assunzioni che NON vanno fissate nei requisiti (sono scelte della fase di progetto, da motivare):

- il **tipo** di interazione di ciascun flusso (evento, dispatch, request/reply): i requisiti non dicono, per esempio, se il sensor *emette eventi* o *invia dispatch*;
- il **trasporto** (MQTT, TCP, CoAP, ...): non e' un requisito che il sensor usi MQTT;
- la **tecnologia** di realizzazione (es. qak/QActor), il cui uso va motivato;
- quale **software del DDR** realizza il cargorobot.

La formalizzazione di questi flussi in un modello concreto e' compito dello Sprint successivo (Analisi del Problema) e del Progetto.

Core Business e Goal del primo prototipo

Domanda al committente: e' corretto concordare che il 'core business' consiste anzitutto nel **portare un container, accettata la richiesta, dall'IOPort allo slot riservato passando per la marcatura in slot5**, mentre la gestione del display, del LED lampeggiante e del 'Out of service' sono aspetti da consolidare in iterazioni successive?

Goal del primo prototipo (fase Development). Realizzare il **cargoservice** che, ricevuta una richiesta valida (pushbutton) con stiva non piena e IOPort libero, riserva uno slot, attende il container all'IOPort (entro il tempo previsto), comanda il cargorobot a portarlo **IOPort → slot5 (marcatura) → slot riservato**, e riporta il robot a HOME, tornando pronto per una nuova richiesta.

Nel primo prototipo si possono semplificare gli output (display/LED come log a video) e si possono **ignorare** *Out of service* e barcode (confermato dal committente), affrontandoli in un'iterazione successiva.

Domande al committente e chiarimenti ricevuti

D1. Posizioni di IOPort, HOME, slot1..slot5?

Chiarimento: un servizio DDR fornito ha gia' una **mappa built-in** dell'area; le coordinate provengono da quella mappa, non sono un dato da inventare.

D2. Come si rileva che l'IOPort e' "occupato da un container"?

(ancora da chiarire: e' lo stesso sensor della presenza, o un'informazione distinta?)

D3. Timeout 30s: cosa succede se scade?

Chiarimento: si assume che il container **NON sia arrivato**: il sistema si disengagia e libera lo slot riservato.

D4. Marcatura: il barcode va memorizzato?

Chiarimento: basta il **segnale** di completamento (markingDone); il barcode in se' non e' gestito dal cargoservice.

D5. Display: che cos'e' e cosa mostra?

Chiarimento: dovrebbe essere una **pagina web**, che **non** include il sensor, e deve mostrare almeno lo **stato corrente degli slot**.

D6. Out of service: le richieste in arrivo?

Chiarimento: vanno **rifiutate** (retrylater) finche' il servizio non torna attivo.

D7. Pushbutton: una sola richiesta per volta?

Chiarimento: si', come da requisito (le altre ricevono retrylater).

D8. LED: dove e' collocato? I requisiti non lo specificano (da chiarire col committente).

Problematiche aperte (per l'analista)

P1 — Realizzazione dei dispositivi senza hardware fisico.

Non disponendo del PicoW fisico, sensor, pushbutton, display, marker e LED vanno realizzati in modo **simulato** (in Python/Java). Cio' che conta a livello di requisiti e' l'**informazione** scambiata (per il sensor: la distanza D), non il dispositivo ne' il trasporto: sostituendo il simulato con l'hardware reale, le informazioni restano le stesse.

P2 — Natura e interazione del sensor dell'IOPort.

I requisiti non dicono **cosa** sia esattamente il sensor, ne' **come** comunichi il dato (evento? dispatch? quale trasporto?): sono decisioni di progetto da motivare. Il sensor va inoltre tenuto distinto dal sonar eventualmente montato sul **robot** (es. in WEnv), che **non riguarda** questa applicazione.

P3 — Movimento IOPort → slot5 → slot.

Il deposito e' modellato in modo astratto: il container si considera spostato quando il cargorobot **raggiunge** la posizione (slot5 e poi lo slot), tramite moverobot(X,Y), senza richiedere un attuatore di presa (non presente nel DDR).

P4 — Test Plan funzionali. Su indicazione del docente ("questo test va formalizzato il prima possibile") il Test Plan e' stato **formalizzato gia' in questa fase**: vedi la sezione *Piano di Test Funzionali* qui sotto.

Piano di Test Funzionali (Sprint0)

Come nel metodo del corso (cfr. la 'mappa mentale' del DDRBoundary), il test e' **automatizzabile**: il cargoservice mantiene lo stato della stiva, che evolve in conseguenza delle azioni; al termine lo stato finale si confronta **programmaticamente** con quello atteso (esito **PASS/FAIL**). Ogni TP indica preconditione, azione, risposta e asserzioni finali.

TP1 CARICO NOMINALE (test significativo per la demo)

precondizione : sistema available; IOPort libero; sonar ok; almeno uno slot libero (es. slot2)

azione : loadrequest() (pushbutton)

risposta att. : answer(reserved(slot2))

sequenza : containerAtIOPort (entro 30s) -> moverobot(IOPort) -> moverobot(slot5)
-> markingDone -> moverobot(slot2) -> moverobot(HOME)

ASSERZIONI (PASS sse tutte vere):

hold.slot("slot2").occupied == true

posizione cargorobot == HOME

stato cargoservice == available

(display == holdstate ; LED == off)

TP2 RIFIUTO PER STIVA PIENA

precondizione : slot1..slot4 tutti occupati

azione : loadrequest()

risposta att. : answer(reject)

ASSERZIONE : stato della stiva INVARIATO ; cargoservice resta available

TP3 RETRYLATER (IOPort occupato OPPURE Out of service)

precondizione : IOPort occupato da un container oppure sistema Out of service

azione : loadrequest()

risposta att. : answer(retrylater)

ASSERZIONE : stato della stiva INVARIATO

TP4 TIMEOUT (container non arrivato)

precondizione : richiesta accettata (slot riservato), sistema engaged

azione : il container NON arriva entro 30s

risultato att.: disengage

ASSERZIONI : slot riservato di nuovo libero ; LED == off ; cargoservice == available

TP5 OUT OF SERVICE (sospetto guasto sonar)

precondizione : sistema in funzione

azione : il sonar misura $D > DFREE$ per $\geq 3s$ -> sonarFail

risultato att.: display == "Out of service" ; le richieste ricevono retrylater

ripristino : $D \leq DFREE$ -> sonarOk -> available

TP1 e' il Test Plan automatizzato significativo richiesto per la demo: copre l'intero ciclo e termina con asserzioni PASS/FAIL integrabili in JUnit o nel log di collaudo.



By Alexander Kreuzer

Email: Alexander.kreuzer@studio.unibo.it

GIT repo: https://github.com/Alexing-uni/ISS26_Alexander_Kreuzer